```
// src/api/RestaurantEndpoints.js
import { get, post, put, destroy, patch } from "./helpers/ApiRequestsHelper";

function getAll() {
  return get("restaurants");
}

function getDetail(id) {
  return get(`restaurants/${id}`);
}

function getRestaurantCategories() {
  return get("restaurantCategories");
}

function create(data) {
  return post("restaurants", data);
}

function update(id, data) {
  return put(`restaurants/${id}`, data);
}

function remove(id) {
  return destroy(`restaurants/${id}`);
}

/* SOLUTION */
function toggleIsUnlimited(id) {
  return patch(`restaurants/${id}/toggleIsUnlimited`);
}

export {
  getAll,
  getDetail,
  getRestaurantCategories,
  create,
  update,
  remove,
  toggleIsUnlimited,
};
```

```
// src/screens/restaurants/RestaurantsScreen.js
import React, { useContext, useEffect, useState } from 'react'
import { StyleSheet, FlatList, Pressable, View } from 'react-native'
import TextRegular from '../../components/TextRegular'
import TextSemiBold from '../../components/TextSemibold'
import { MaterialCommunityIcons } from '@expo/vector-icons'
import * as GlobalStyles from '../../styles/GlobalStyles'
import { AuthorizationContext } from '../../context/AuthorizationContext'
import { showMessage } from 'react-native-flash-message'
import { getAll, remove, toggleIsUnlimited } from '../../api/RestaurantEndpoints'
import ImageCard from '../../components/ImageCard'
import DeleteModal from '../../components/DeleteModal'

export default function RestaurantsScreen({ navigation, route }) {
const [restaurants, setRestaurants] = useState([])
```

```
const [restaurantToBeDeleted, setRestaurantToBeDeleted] = useState(null)
const { loggedInUser } = useContext(AuthorizationContext)

useEffect(() => {
if (loggedInUser) {
fetchRestaurants()
} else {
setRestaurants(null)
}
}, [loggedInUser, route])

const fetchRestaurants = async () => {
try {
const fetchedRestaurants = await getAll()
setRestaurants(fetchedRestaurants)
} catch (error) {
showMessage({
message: `There was an error while retrieving restaurants. ${error} `,
type: 'error',
style: GlobalStyles.flashStyle,
titleStyle: GlobalStyles.flashTextStyle
})
}
}

const removeRestaurant = async (restaurant) => {
try {
await remove(restaurant.id)
await fetchRestaurants()
setRestaurantToBeDeleted(null)
showMessage({
message: `Restaurant ${restaurant.name} succesfully removed`,
type: 'success',
style: GlobalStyles.flashStyle,
titleStyle: GlobalStyles.flashTextStyle
})
} catch (error) {
console.log(error)
setRestaurantToBeDeleted(null)
showMessage({
message: `Restaurant ${restaurant.name} could not be removed.`,
type: 'error',
style: GlobalStyles.flashStyle,
titleStyle: GlobalStyles.flashTextStyle
})
}
}

/* SOLUTION */
const handleToggleUnlimited = async (restaurant) => {
try {
await toggleIsUnlimited(restaurant.id)
await fetchRestaurants()
showMessage({
message: `Estado ilimitado actualizado para ${restaurant.name}`,
type: 'success',
```

```
style: GlobalStyles.flashStyle,
titleStyle: GlobalStyles.flashTextStyle
})
} catch (error) {
console.log(error)
showMessage({
message: `Error: Un propietario no puede tener más de 3 restaurantes ilimitados.`,
type: 'error',
style: GlobalStyles.flashStyle,
titleStyle: GlobalStyles.flashTextStyle
})
}
}

const renderRestaurant = ({ item }) => {
return (
<ImageCard
imageUri={item.logo ? { uri: process.env.API_BASE_URL + '/' + item.logo } : undefined}
title={item.name}
onPress={() => {
navigation.navigate('RestaurantDetailScreen', { id: item.id })
}}
>
{item.description}
{item.averageServiceMinutes !== null &&
Avg. service time: <TextSemiBold textStyle={{ color: GlobalStyles.brandPrimary }}>{item.averageServiceMinu
}
Shipping: <TextSemiBold textStyle={{ color: GlobalStyles.brandPrimary }}>{item.shippingCosts} €

    {/* SOLUTION */}
    <View style={{ marginTop: 10 }}>
      <TextRegular>
        Pedidos (últimas 2h): <TextSemiBold textStyle={{ color: GlobalStyles.brandPrimary }}>{item.ordersI
      </TextRegular>
      {item.isClosedByLimit && !item.isUnlimited && (
        <View style={styles.saturadoBadge}>
          <TextSemiBold textStyle={{ color: 'white' }}>Saturado</TextSemiBold>
        </View>
      )}
    </View>

    <View style={styles.actionButtonsContainer}>
      <Pressable
        onPress={() => navigation.navigate('EditRestaurantScreen', { id: item.id })}
        style={({ pressed }) => [
          {
            backgroundColor: pressed
              ? GlobalStyles.brandBlueTap
              : GlobalStyles.brandBlue
          },
          styles.actionButton
        ]}>
        <View style={[{ flex: 1, flexDirection: 'row', justifyContent: 'center' }]}>
          <MaterialCommunityIcons name='pencil' color={'white'} size={20} />
          <TextRegular textStyle={styles.text}>
            Edit
```

```
          </TextRegular>
        </View>
      </Pressable>

      <Pressable
        onPress={() => { setRestaurantToBeDeleted(item) }}
        style={({ pressed }) => [
          {
            backgroundColor: pressed
              ? GlobalStyles.brandPrimaryTap
              : GlobalStyles.brandPrimary
          },
          styles.actionButton
        ]}>
        <View style={[{ flex: 1, flexDirection: 'row', justifyContent: 'center' }]}>
          <MaterialCommunityIcons name='delete' color={'white'} size={20} />
          <TextRegular textStyle={styles.text}>
            Delete
          </TextRegular>
        </View>
      </Pressable>

      {/* SOLUTION */}
      <Pressable
        onPress={() => handleToggleUnlimited(item)}
        style={({ pressed }) => [
          {
            backgroundColor: pressed
              ? (item.isUnlimited ? GlobalStyles.brandSecondaryTap : GlobalStyles.brandSuccessTap)
              : (item.isUnlimited ? GlobalStyles.brandSecondary : GlobalStyles.brandSuccess)
          },
          styles.actionButton
        ]}>
        <View style={[{ flex: 1, flexDirection: 'row', justifyContent: 'center' }]}>
          <MaterialCommunityIcons name={item.isUnlimited ? 'infinity-off' : 'infinity'} color={'white'} si
          <TextRegular textStyle={styles.text}>
            {item.isUnlimited ? 'Limited' : 'Unlimited'}
          </TextRegular>
        </View>
      </Pressable>
    </View>
  </ImageCard>
)

}

const renderEmptyRestaurantsList = () => {
return (

No restaurants were retrieved. Are you logged in?

)
}

const renderHeader = () => {
return (
```

```
<>
{loggedInUser &&
<Pressable
onPress={() => navigation.navigate('CreateRestaurantScreen')
}
style={({ pressed }) => [
{
backgroundColor: pressed
? GlobalStyles.brandGreenTap
: GlobalStyles.brandGreen
},
styles.button
]}>
<View style={[{ flex: 1, flexDirection: 'row', justifyContent: 'center' }]}>
<MaterialCommunityIcons name='plus-circle' color={'white'} size={20} />

Create restaurant



}
</>
)
}

return (
<>
<FlatList
style={styles.container}
data={restaurants}
renderItem={renderRestaurant}
keyExtractor={item => item.id.toString()}
ListHeaderComponent={renderHeader}
ListEmptyComponent={renderEmptyRestaurantsList}
/>
<DeleteModal
isVisible={restaurantToBeDeleted !== null}
onCancel={() => setRestaurantToBeDeleted(null)}
onConfirm={() => removeRestaurant(restaurantToBeDeleted)}>
The products of this restaurant will be deleted as well

</>
)
}

const styles = StyleSheet.create({
container: {
flex: 1
},
button: {
borderRadius: 8,
height: 40,
marginTop: 12,
padding: 10,
alignSelf: 'center',
flexDirection: 'row',
```

```
width: '80%'
},
actionButton: {
borderRadius: 8,
height: 40,
marginTop: 12,
margin: '1%',
padding: 10,
alignSelf: 'center',
flexDirection: 'column',
width: '31%'
},
actionButtonsContainer: {
flexDirection: 'row',
bottom: 5,
position: 'absolute',
width: '90%'
},
text: {
fontSize: 16,
color: 'white',
alignSelf: 'center',
marginLeft: 5
},
emptyList: {
textAlign: 'center',
padding: 50
},
/* SOLUTION */
saturadoBadge: {
backgroundColor: GlobalStyles.brandPrimary,
borderRadius: 5,
paddingHorizontal: 8,
paddingVertical: 4,
alignSelf: 'flex-start',
marginTop: 5
}
})
```